

XDP-L2-Guard: Techniczny podręcznik referencyjny

Wysokowydajna orkiestracja ruchu w warstwie 2 i 3 za pomocą eBPF/XDP

Krzysztof Taraszkiewicz

Józef Sztabiński

June 24, 2026

Abstract

XDP-L2-Guard to wysokowydajny, programowalny filtr orkiestracji sieciowej wykorzystujący mechanizm Linux eXpress Data Path (XDP). Działając bezpośrednio w punkcie podpięcia (hook) sterownika karty sieciowej (NIC), zapewnia możliwość przetwarzania pakietów w czasie poniżej mikrosekundy dla ruchu w warstwie 2 i 3. Umożliwia to odrzucanie pakietów z prędkością łącza (wire-speed), przekierowywanie pakietów, hairpinning oraz translację adresów sieciowych (NAT) ze śledzeniem stanu przed alokacją buforów gniazd (`sk_buff`) przez jądro Linuksa. Niniejszy podręcznik służy jako wyczerpujące, niskopoziomowe odniesienie techniczne obejmujące struktury pamięci na poziomie języka C, zarządzanie CO-RE w języku Python, algorytmy sum kontrolnych oraz strategię wdrażania.

Contents

1	eBPF a tradycyjne moduły jądra Linuksa	1
1.1	Metoda działania i uzyskiwanie uprawnień	1
1.2	Kluczowe zalety eBPF w porównaniu do modułów LKM	2
2	Architektura systemu i paradygmat	2
2.1	Schemat komponentów architektury	3
3	Implementacja warstwy danych (eBPF/XDP)	3
3.1	Układ pamięci i walidacja wskaźników	3
3.2	Schemat mapy akcji (Action Map Schema)	4
3.3	Niskopoziomowa semantyka akcji	4
3.3.1	XDP_DROP	4
3.3.2	XDP_TX	4
3.3.3	XDP_REDIRECT & ACTION_NAT (DNAT)	5
4	Warstwa sterowania: orkiestracja i odpytywanie	5
4.1	Silnik ładujący (<code>loader.py</code>)	5
4.2	Interfejs CLI zarządzania (<code>guard.py</code> & <code>guard.sh</code>)	5
5	Prezentacja działania i przykłady testowe	6
5.1	Konfiguracja topologii sieciowej	6
5.2	Stosowanie akcji	6
6	Operacyjny DevOps i diagnostyka	7
6.1	Strojenie wydajności i rzeczywiste limity	7
6.1.1	Studium przypadku: Testowanie trybu natywnego kart sieciowych Intel	7
6.2	Matryca diagnostyczna	8

7	Badanie potencjału technologii Carrier Ethernet za pomocą XDP	8
7.1	Testowanie wewnętrznych możliwości routingu	8
7.2	Sprzętowe odciążenie i karty sieciowe Intel	9
8	Testowanie koncepcji przełącznika programowego	9
8.1	eBPF jako alternatywa dla iptables	9
8.2	Podstawowy NAT i potencjał warstwy danych	9
9	Podsumowanie projektu i wnioski	10
9.1	Wrażenia i wyciągnięte wnioski	10
9.2	Dlaczego ten projekt był wart realizacji	10
9.3	Kierunki przyszłych badań i testów	10

1 eBPF a tradycyjne moduły jądra Linuksa

Historycznie rzecz biorąc, modyfikowanie lub rozszerzanie zachowania jądra Linuksa wymagało napisania modułu jądra Linuksa (LKM – Linux Kernel Module) lub rekompilacji kodu źródłowego jądra. Chociaż moduły LKM mają pełne uprawnienia i nieograniczony dostęp do pamięci jądra oraz przepływów wykonania, ze swojej natury niosą ze sobą poważne ryzyko. Błąd w module LKM, taki jak dereferencja wskaźnika zerowego lub nieskończona pętla, może natychmiast doprowadzić do paniki jądra (kernel panic), powodując awarię całego systemu operacyjnego. Ponadto moduły LKM stanowią zagrożenie dla bezpieczeństwa, ponieważ działają z nieograniczonymi uprawnieniami Ring 0 bez wbudowanej izolacji.

eBPF (Extended Berkeley Packet Filter) wprowadza zasadniczo inny paradygmat działania. Umożliwia programistom bezpieczne wykonywanie niestandardowego kodu bajtowego (byte-code) wewnątrz jądra bez zmiany kodu źródłowego jądra lub ładowania standardowych modułów.

1.1 Metoda działania i uzyskiwanie uprawnień

W przeciwieństwie do tradycyjnego modułu jądra, który jest konsolidowany bezpośrednio z symbolami jądra i wykonywany natywnie jako kod jądra, programy eBPF są wstrzykiwane do jądra za pomocą wywołania systemowego `bpf()` z przestrzeni użytkownika. Zanim jednak jakikolwiek program eBPF zostanie dopuszczony do wykonania, musi przejść przez rygorystyczny **weryfikator jądra (Kernel Verifier)**.

Weryfikator przeprowadza statyczną analizę kodu bajtowego eBPF, aby zagwarantować, że program jest bezpieczny do uruchomienia. Zapewnia on, że:

- Nie ma nieskończonych pętli (wszystkie programy muszą się zakończyć).
- Wszystkie dostępy do pamięci są ściśle kontrolowane pod kątem granic zakresu (bounds-checked).
- Nie są używane żadne niezainicjalizowane zmienne.
- Rozmiar programu nie przekracza rygorystycznych limitów złożoności.

Gdy weryfikator zatwierdzi kod bajtowy, **kompilator JIT (Just-In-Time) eBPF** tłumaczy kod bajtowy na natywne instrukcje maszynowe (np. `x86_64` lub `ARM64`) specyficzne dla sprzętu hosta. Ten skompilowany przez JIT kod działa następnie z prędkością bliską natywnej bezpośrednio w przestrzeni jądra.

W ten sposób eBPF uzyskuje uprawnienia jądra nie poprzez omijanie warstw bezpieczeństwa, ale poprzez matematyczne udowodnienie jądra, że jego wykonanie jest bezpieczne. Nie może on uzyskać dostępu do dowolnej pamięci jądra; jest ściśle ograniczony do struktur danych specyficznych dla kontekstu (takich jak struktura `xdp_md` dla danych pakietów) oraz predefiniowanych funkcji pomocniczych jądra (`bpf_helpers`), które zapewniają bezpieczne API do interakcji z szerszym systemem.

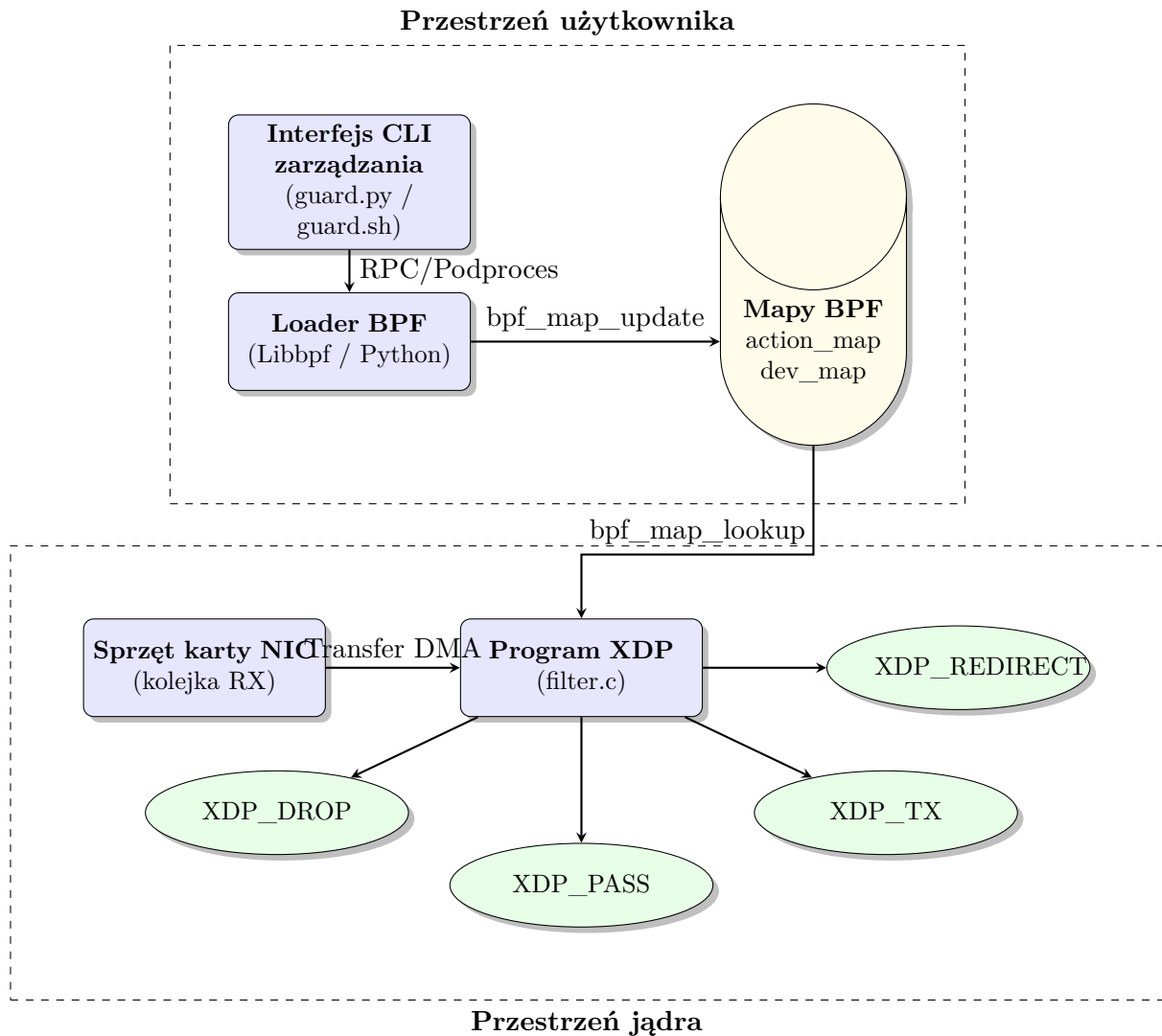
1.2 Kluczowe zalety eBPF w porównaniu do modułów LKM

- **Bezpieczeństwo i stabilność:** Rygorystyczny proces weryfikacji gwarantuje, że program eBPF nie doprowadzi do awarii systemu. Jeśli kod narusza granice pamięci lub jego wykonanie trwa zbyt długo, weryfikator po prostu odrzuca go w momencie ładowania.
- **Dynamiczne ładowanie i aktualizacje:** Programy eBPF mogą być ładowane, aktualizowane i odinstalowywane w locie, bez restartowania systemu czy przerywania pracy sieci. Umożliwia to dynamiczne i płynne aktualizacje reguł filtrowania pakietów.
- **Wysoka wydajność:** Dzięki kompilacji JIT programy eBPF działają z prędkościami natywnymi. W kontekście XDP pozwala to na przetwarzanie pakietów bezpośrednio w sterowniku karty sieciowej (NIC), zanim jądro w ogóle przydzieli pamięć (`sk_buff`), co zapewnia opóźnienie poniżej mikrosekundy.
- **Bezpieczeństwo i izolacja:** Ponieważ eBPF jest ograniczony do określonych punktów podpięcia (hooks) i funkcji pomocniczych, drastycznie zmniejsza obszar podatności na ataki w porównaniu z modułami LKM, które mają absolutną kontrolę nad systemem.

2 Architektura systemu i paradygmat

System jest ściśle zaprojektowany wokół paradygmatu separacji warstwy sterowania (control plane) od warstwy danych (data plane). **Warstwa danych (Data Plane)** jest napisana w ograniczonym języku C, skompilowana za pomocą Clang/LLVM do kodu bajtowego eBPF i wykonywana bezpiecznie w kontekście jądra. **Warstwa sterowania (Control Plane)** wykorzystuje język Python 3 do orkiestracji cyklu życia programu eBPF, używając narzędzi `iproute2` oraz `bpftool` do zarządzania asynchronicznymi aktualizacjami stanu.

2.1 Schemat komponentów architektury



3 Implementacja warstwy danych (eBPF/XDP)

Kod warstwy danych (`src/data_plane/filter.c`) podłącza się pod pętlę odpytywania NAPI karty sieciowej (NIC) poprzez makro `SEC("xdp")`. Funkcja `xdp_drop_logic` pełni rolę głównego koordynatora pakietów wejściowych (ingress).

3.1 Układ pamięci i walidacja wskaźników

Programy eBPF działają w zweryfikowanym środowisku. Zanim weryfikator zezwoli na dostęp do pamięci pakietu, kod musi udowodnić, że dostęp w granicach pamięci jest bezpieczny. Program otrzymuje kontekst `struct xdp_md`, dostarczający wskaźniki mapowane przez DMA:

```
void *data = (void *) (long) ctx->data;  
void *data_end = (void *) (long) ctx->data_end;
```

Używamy niestandardowego makra `BOUNDS_CHECK` zdefiniowanego w pliku `headers.h`:

```
#define BOUNDS_CHECK(pointer, type, data_end) \
do { \
    if ((void *)(pointer) + sizeof(type) > (void *)(data_end)) \
        return XDP_PASS; \
} while(0)
```

Potok analizy (parsing pipeline) przebiega ściśle warstwa po warstwie:

1. **Warstwa 2 (Ethernet):** Rzutowanie `data` na strukturę `struct ethhdr`. Weryfikacja granic pamięci. Sprawdzenie, czy `eth->h_proto == bpf_htons(ETH_P_IP)`. Jeśli nie, przekazanie pakietu do jądra poprzez `XDP_PASS`.
2. **Warstwa 3 (IPv4):** Rzutowanie `data + sizeof(struct ethhdr)` na strukturę `struct iphdr`. Weryfikacja granic pamięci. Wyodrębnienie docelowego adresu IP (`ip->daddr`).

3.2 Schemat mapy akcji (Action Map Schema)

Wyodrębniony adres IP jest używany jako klucz wyszukiwania w głównej mapie haszującej BPF, `action_map`. Mapa oczekuje adresu IPv4 w sieciowej kolejności bajtów (`__u32`) i zwraca strukturę `struct action_cfg`:

```
struct action_cfg {
    __u32 target;           // 0: PASS, 1: DROP, 2: TX, 3: REDIRECT, 4:
    NAT
    __u32 new_ip;           // Docelowy adres IP dla scenariuszy NAT
    __u32 ifindex;          // Indeks interfejsu systemu operacyjnego
    dla REDIRECT
    __u64 dropped_packets;  // Atomowy licznik wydajności
};
```

3.3 Niskopoziomowa semantyka akcji

W zależności od pobranej wartości `target`, jądro wykonuje jedną z następujących procedur:

3.3.1 XDP_DROP

Najprostsza i najszybsza akcja. Sterownik karty sieciowej (NIC) odrzuca ramkę i natychmiast odzyskuje bufor DMA. Kod atomowo inkrementuje globalną mapę tablicy `total_dropped` oraz specyficzną dla reguły mapę `dropped_packets`.

3.3.2 XDP_TX

`XDP_TX` odsyła pakiet z powrotem przez ten sam interfejs, przez który dotarł. W naszej implementacji jest to wykorzystywane do zaawansowanej manipulacji ICMP. W przypadku wykrycia pakietu `ICMP_ECHO` (Ping):

1. Typ jest nadpisywany jako `ICMP_ECHOREPLY` (0).
2. **Obliczenia sumy kontrolnej:** Ponieważ Typ 8 zmienił się na Typ 0, 16-bitowa suma kontrolna dopełnienia do jedynki musi być przyrostowo zaktualizowana przez dodanie wartości `0x0800`.
3. Adresy MAC i IP są zamieniane miejscami przy użyciu `__builtin_memcpy` w celu uzyskania optymalnej kompilacji wbudowanej (inline).

3.3.3 XDP_REDIRECT & ACTION_NAT (DNAT)

ACTION_NAT przeprowadza translację docelowego adresu sieciowego (Destination Network Ad-

dress Translation – DNAT) bezpośrednio na poziomie sprzętowym.

```
__u32 old_daddr = ip->daddr;
__u32 new_daddr = cfg->new_ip;
__u32 csum = ~bpf_ntohs(ip->checksum);
csum += (~old_daddr & 0xFFFF) & 0xFFFF;
csum += (~old_daddr >> 16) & 0xFFFF;
while (csum >> 16) csum = (csum & 0xFFFF) << 16;
ip->checksum = bpf_htons(csum);
```

Ta logika bitowa przyrostowo aktualizuje sumę kontrolną nagłówka IP bez potrzeby pełnego przeliczania pakietu, zapewniając opóźnienia poniżej mikrosekundy. XDP_REDIRECT działa według dokładnie tej samej logiki, kończąc się jednak przekazaniem ramki do interfejsu wskazanego w dev_map.

4 Warstwa sterowania: orkiestracja i odpytywanie

Warstwa sterowania (Control Plane) składa się z dwóch głównych skryptów Pythona: loader.py do zarządzania cyklem życia oraz guard.py do definiowania reguł za pomocą interfejsu wiersza poleceń (CLI).

4.1 Silnik ładujący (loader.py)

Loader obsługuje kompilację typu Ahead-Of-Time (AOT) oraz podłączanie do jądra za pomocą zestawu narzędzi iproute2.

- **Kompilacja:** Uruchamia make w celu skompilowania pliku filter.c do obiektu ELF filter.o.
- **Wybór trybu:** Wykrywa, czy podmontować program w wysokowydajnym trybie natywnym (xdpdrv), czy w ogólnym trybie programowym (xdpgeneric).
- **Podłączanie:** Używa polecenia `ip -force link set dev <interface> xdpdrv obj filter.o sec xdp`. Flaga `-force` gwarantuje atomowe nadpisanie nieaktywnych ("zombie") programów.
- **Asynchroniczne odpytywanie:** Działa w nieskończonej pętli (`time.sleep(1)`), która wywołuje `bpftool -j map dump name action_map`. Dekoduje dane wyjściowe JSON, rozpakowuje strumień bajtów i tłumaczy surowe metryki na czytelne dla człowieka logi.

4.2 Interfejs CLI zarządzania (guard.py & guard.sh)

Projekt udostępnia dwa identyczne interfejsy przypominające iptables do manipulowania aktywnymi mapami BPF za pomocą narzędzia bpftool:

- **src/control_plane/guard.py:** Główna implementacja w języku Python z użyciem biblioteki argparse.
- **scripts/guard.sh:** Alternatywa w czystym Bashu wykorzystująca awk i sed dla środowisk, w których Python jest niedostępny lub niepożądany.

Oba skrypty współdzielą dokładnie tę samą składnię i argumenty.

Przykłady składni CLI:

```
# Dodaj standardowa regule DROP dla zlosliwego hosta (wersja Python)
sudo python3 guard.py --append --destination 192.168.1.50 --jump DROP

# Ustanow translacje NAT na poziomie sprzetowym (wersja Bash)
sudo ./guard.sh -A -d 10.0.0.100 -j NAT --to-destination 10.0.0.250

# Przekieruj ruch przychodzacy dla adresu 10.0.0.5 na inny wirtualny
  Ethernet
sudo ./guard.sh -A -d 10.0.0.5 -j REDIRECT --oif veth1

# Wyświetl liste wszystkich aktywnych regul dynamicznie pobranych z
  pamieci jadra
sudo python3 guard.py --list
```

Wewnętrznie oba skrypty rozpoznają interfejsy systemowe (np. `veth1`) do całkowitoliczbowych wartości `ifindex` za pośrednictwem `/sys/class/net/<iface>/ifindex` i formatują tablice bajtów struktur C (albo za pomocą `struct.pack` w Pythonie, albo `printf` w Bashu), aby wstrzyknąć je do jądra poprzez polecenie `bpftool map update`.

5 Prezentacja działania i przykłady testowe

Aby zademonstrować i zweryfikować możliwości systemu XDP-L2-Guard, katalog `scripts/example/` udostępnia interaktywne środowisko testowe wykorzystujące sieciowe przestrzenie nazw Linuksa (`netns`).

5.1 Konfiguracja topologii sieciowej

Skrypt `setup.sh` inicjalizuje symulowane 3-węzłowe środowisko sieciowe:

- **ns1 (Filtr/Router):** Przypisany adres IP 10.0.0.1. Obiekt eBPF jest podłączony do wirtualnych interfejsów `v1-2` i `v1-3`. Włączone jest przekazywanie pakietów IPv4 (`forwarding`).
- **ns2 (Nadawca):** Przypisany adres IP 10.0.0.2. Działa jako źródło ruchu.
- **ns3 (Cel):** Przypisany adres IP 10.0.0.3 w trybie nasłuchiwanie (`promiscuous mode`) w celu obserwowania przekierowanych pakietów.

5.2 Stosowanie akcji

Skrypt `set_action.sh` używa skryptu `guard.sh` do dynamicznego nakładania reguł na ruch pochodzący z węzła `ns2`:

```
# Terminal 1: Uruchom konfiguracje i utrzymaj aktywny filtr XDP
sudo ./setup.sh

# Terminal 2: Testuj rozne akcje XDP w czasie rzeczywistym
./set_action.sh drop      # Ruch z ns2 jest po cichu odrzucany
./set_action.sh redirect  # Ruch z ns2 jest przekierowywany do ns3
./set_action.sh tx        # Hairpin ICMP echo (Ping odpowiada
  natychmiast)
./set_action.sh nat        # Docelowy IP jest przepisywany na 10.0.0.3
```

Ta konfiguracja udowadnia, że złożone operacje warstwy 2 (`REDIRECT`) oraz warstwy 3 (`NAT`) zachodzą w całości w warstwie eBPF, całkowicie omijając konwencjonalną logikę routingu w `ns1`.

6 Operacyjny DevOps i diagnostyka

6.1 Strojenie wydajności i rzeczywiste limity

Aby osiągnąć teoretyczną maksymalną przepustowość (10-40 Gbps), konieczne jest przeprowadzenie kilku optymalizacji na poziomie sprzętowym.

- **Tryb natywny (xdpdrv) a tryb ogólny (xdpgeneric):** Tryb natywny uruchamia program eBPF bezpośrednio w sterowniku karty sieciowej (NIC), zanim jądro przydzieli jakąkolwiek pamięć. Tryb ogólny działa na dalszym etapie stosu. Tryb natywny jest kluczowy dla ochrony przed zalewem pakietów (flood protection).
- **Wyłączenie sprzętowych odciążen (Offloads):** Odciążenia takie jak Generic Receive Offload (GRO) agregują pakiety zanim zostaną one zauważone przez XDP, co powoduje ominięcie filtra. Wyłączenie następuje poprzez: `sudo ethtool -K eth0 gro off gso off rxvlan off`.
- **Przypinanie rdzeni CPU (CPU Pinning):** Należy upewnić się, że przerwania IRQ dla XDP są przypięte do dedykowanych fizycznych rdzeni procesora za pomocą `smp_affinity`.

6.1.1 Studium przypadku: Testowanie trybu natywnego kart sieciowych Intel

Podczas rzeczywistych testów obciążeniowych wykorzystujących wiele niezależnych maszyn atakujących, które zalewały pakietami interfejs sieciowy Intel, naszym celem było wykorzystanie natywnego trybu XDP (`xdpdrv`) dla uzyskania maksymalnego przyspieszenia sprzętowego. Ostatecznie napotkaliśmy wyzwania związane z niskopoziomą konfiguracją sprzętową, a nie limity obliczeniowe procesora hosta.

Choć w tym konkretnym teście nie byliśmy w stanie idealnie dopasować konfiguracji sterownika karty sieciowej w celu osiągnięcia przetwarzania z prędkością łącza (wire-speed), uznajemy to za kluczowy obszar przyszłego rozwoju. Jesteśmy w pełni świadomi ogromnego potencjału wydajnościowego, jaki oferuje natywne wykonywanie na poziomie sprzętowym. Byliśmy wyjątkowo blisko działającej natywnej implementacji, a rozwiązanie tych specyficznych dla sprzętu niuansów konfiguracyjnych pozostaje priorytetowym kamieniem milowym dla kolejnej ewolucji XDP-L2-Guard.

6.2 Matryca diagnostyczna

Objaw	Analiza przyczyny źródłowej	Rozwiązanie
<code>libbpf: load fail</code>	Weryfikator eBPF odrzucił granice kodu.	Sprawdź <code>dmesg</code> . Upewnij się, że <code>BOUND_CHECK</code> obejmuje całą odczytywaną pamięć.
Reguły nałożone, brak odrzucania	Podłączono do niewłaściwego interfejsu lub trybu.	Sprawdź za pomocą <code>ip link show</code> . Upewnij się, że XDP jest podłączony do interfejsu fizycznego, a nie do mostka (bridge).
EBUSY podczas podłączania	Inny program XDP jest już załadowany.	Użyj flagi <code>-force</code> w poleceniu <code>ip link set dev <iface> xdp obj ...</code>
Wysokie zużycie CPU przez SoftIRQ	Blokowanie zasobów (lock contention) na mapie <code>total_dropped</code> .	Zrefaktoryzuj mapę do <code>BPF_MAP_TYPE_PERCPU_ARRAY</code> .
Pakiety omijają filtr	Aktywne jest sprzętowe odciążenie (offloading).	Wyłącz GRO/LRO za pomocą <code>ethtool -K <iface> gro off</code> .
Błąd tworzenia mapy BPF	Limit <code>RLIMIT_MEMLOCK</code> jest zbyt niski.	Zwiększ limity pamięci: <code>ulimit -l unlimited</code> .
Ciche niepowodzenie <code>XDP_TX</code>	Niepoprawne ponowne obliczenie sumy kontrolnej.	Zweryfikuj matematykę dopełnienia do jedynki. Upewnij się, że logika zamiany adresów MAC jest poprawna.

7 Badanie potencjału technologii Carrier Ethernet za pomocą XDP

Choć standardowa dystrybucja Linuksa nie jest zazwyczaj stosowana jako rdzeniowy router Carrier Ethernet, nowoczesne możliwości jądra dostarczają elementów konstrukcyjnych do testowania tych koncepcji. Carrier Ethernet opiera się na dostarczaniu pakietów o wysokiej przepustowości oraz wewnętrznych mechanizmach etykietowania (takich jak MPLS czy VLAN) w celu efektywnego kierowania ruchem. Nasz projekt, XDP-L2-Guard, służy jako małe środowisko testowe do zrozumienia potencjału eBPF i XDP w tego typu scenariuszach.

7.1 Testowanie wewnętrznych możliwości routingu

Historycznie routing oparty na etykietach MPLS w systemie Linux opiera się na standardowym stosie sieciowym jądra (poprzez moduły takie jak `mpls_router`), co wprowadza narzut na procesor. Nasza implementacja w pliku `filter.c` skupia się obecnie na podstawowym wyszukiwaniu adresów docelowych IPv4, ale skutecznie demonstruje podstawową zasadę wymaganą dla routera LSR (Label Switch Router).

Działając w punkcie podłączenia XDP, przechwytujemy pakiety przed alokacją `sk_buff` przez jądro. Przetestowana przez nas funkcjonalność `XDP_REDIRECT` pokazuje, że stworzenie płaszczyzny danych szybkiej ścieżki (fast-path data plane) jest całkowicie możliwe. Choć nie

zaimplementowaliśmy pełnego parsowania MPLS, nasz kod udowadnia potencjał: logika eBPF mogłaby łatwo zostać rozszerzona o odczytywanie etykiet MPLS bezpośrednio ze wskaźników `data` i natychmiastowe kierowanie pakietów za pomocą mapy `dev_map`.

7.2 Sprzętowe odciążenie i karty sieciowe Intel

Przetestowaliśmy XDP-L2-Guard w celu zrozumienia natywnego trybu XDP (`xdpdrv`) na nowoczesnych kartach sieciowych Intel (takich jak serie 700 oraz 800 igrice). Teoretycznie, wykonywanie kodu bajtowego eBPF bezpośrednio w sterowniku kolejki odbiorczej (RX queue) karty sieciowej oferuje wydajność porównywalną z DPDK (Data Plane Development Kit – standardowy w branży szkielet programistyczny osiągający wysokie prędkości poprzez całkowite ominięcie jądra systemu operacyjnego na rzecz przetwarzania pakietów w przestrzeni użytkownika), bez konieczności opuszczania środowiska jądra Linuxa.

Mimo że nasz projekt jest jedynie dowodem koncepcji (proof-of-concept), podkreśla on, jak akceleracja sprzętowa (jak Intel Flow Director) połączona z XDP może inteligentnie dystrybuować i przetwarzać ruch przy ekstremalnie niskich opóźnieniach. Nasze eksperymenty potwierdzają teoretyczny potencjał użycia systemu Linux i sprzętu Intel do przetwarzania ruchu klasy operatorskiej (Carrier-grade).

8 Testowanie koncepcji przełącznika programowego

Jednym z głównych celów projektu XDP-L2-Guard było zbadanie, w jaki sposób eBPF może służyć jako podstawa dla programowalnego przełącznika programowego (software switch) i jak wypada w porównaniu z tradycyjnymi narzędziami zaporowymi (firewall).

8.1 eBPF jako alternatywa dla iptables

Tradycyjnie filtrowanie pakietów i NAT w systemie Linux opierają się na narzędziu `iptables`. Jednak `iptables` wykorzystuje model listy sekwencyjnej (złożoność $O(n)$), co oznacza, że wydajność spada wraz ze wzrostem liczby reguł. Nasz projekt pozwolił nam praktycznie przetestować korzyści wydajnościowe wynikające z map eBPF.

Dzięki zaimplementowaniu mapy `action_map` (`BPF_MAP_TYPE_HASH`), nasz skrypt XDP osiąga stały czas wyszukiwania $O(1)$. Chociaż nasza warstwa sterowania w Pythonie (`guard.py`) zarządza jedynie ułamkiem reguł, jakie mogłoby stosować rzeczywiste przedsiębiorstwo, zademonstrowany przez nas podstawowy mechanizm skaluje się bez wysiłku, udowadniając wyższość eBPF nad przestarzałym `iptables`.

8.2 Podstawowy NAT i potencjał warstwy danych

Zamiast polegać na stosie netfilter jądra, zaimplementowaliśmy podstawową akcję `ACTION_NAT` w języku C, aby przetestować bezpośrednią manipulację pakietami. Nasz kod pomyślnie odczytuje docelowe adresy IP, nadpisuje je i ponownie oblicza sumę kontrolną dopełnienia do jedynki przy użyciu operacji bitowych bezpośrednio w pliku `filter.c`.

W połączeniu z `XDP_REDIRECT` tworzy to podstawowy, ale bardzo szybki przełącznik programowy. Choć XDP-L2-Guard nie jest kompletnym, gotowym do wdrożenia produkcyjnego routerem, z powodzeniem demonstruje niewielką część możliwości eBPF, potwierdzając, że architektura Linuxa ma potencjał do obsługi złożonego, szybkiego routingu i etykietowania na najniższym możliwym poziomie oprogramowania.

9 Podsumowanie projektu i wnioski

9.1 Wrażenia i wyciągnięte wnioski

Realizacja projektu XDP-L2-Guard dostarczyła bezcennej, praktycznej wiedzy dotyczącej najniższych poziomów stosu sieciowego systemu Linux. Naszym głównym wrażeniem jest ogromna elastyczność i możliwości połączone z trudną ścieżką nauki. eBPF i XDP oferują bezprecedensową moc przechwytywania i modyfikowania pakietów z prędkością łącza, ale ta moc wiąże się z ograniczeniami narzuconymi przez rygorystyczny weryfikator oraz skomplikowaną obsługą pamięci na poziomie języka C. Pomyślnie udowodniliśmy, że jesteśmy w stanie napisać funkcjonalną warstwę danych zdolną do parsowania nagłówków IP, przeliczania sum kontrolnych oraz orkiestracji podstawowych operacji NAT lub przekierowań – a wszystko to zanim jądro przydzieli jakiegokolwiek standardowe bufory gniazd.

9.2 Dlaczego ten projekt był warty realizacji

Ten projekt był niezwykle wartościowy, ponieważ odmitologizował mechanizmy stojące za nowoczesnymi sieciami chmurowymi (cloud-native networking) oraz sieciami definiowanymi programowo (SDN). Budując XDP-L2-Guard od zera, zmieniliśmy nasze postrzeganie systemu Linux z prostego punktu końcowego lub podstawowej zatory sieciowej w wysoce programowalny, zbliżony do sprzętowego przełącznik. Pozwoliło nam to w praktyce doświadczyć ograniczeń starszych narzędzi, takich jak `iptables`, i w pełni zrozumieć, dlaczego branża tak szybko przechodzi na eBPF. Wiedza zdobyta w zakresie natywnego XDP, sprawdzania granic pamięci oraz rozdzielania warstwy sterowania od warstwy danych stanowi kluczowy fundament współczesnej inżynierii sieciowej.

9.3 Kierunki przyszłych badań i testów

Choć XDP-L2-Guard służy jako udany dowód koncepcji (proof-of-concept), istnieje wiele dróg potencjalnej przyszłej eksploracji:

- **Głębokie parsowanie warstwy 2/etykiet:** Rozszerzenie kodu `filter.c` w celu natywnego parsowania, dodawania (push) i zdejmowania (pop) etykiet MPLS lub tagów VLAN, aby zasymulować kompletny router Carrier Ethernet LSR (Label Switch Router).
- **Zaawansowane strojenie sprzętowe:** Dalsze rozwiązywanie niskopoziomowych niuansów konfiguracji sprzętowej kart sieciowych Intel, aby w pełni odblokować tryb natywny (`xdpdrv`) i porównać go z rozwiązaniami opartymi na DPDK przy przepustowościach rzędu 40+ Gbps.
- **Śledzenie stanu (Stateful Tracking):** Przejście od bezstanowego DNAT do w pełni stanowych połączeń poprzez zaimplementowanie mechanizmu śledzenia połączeń (contrack) z wykorzystaniem map haszujących eBPF LRU (Least Recently Used).
- **Wsparcie dla IPv6:** Rozszerzenie bazy kodu C oraz logiki sprawdzania granic pamięci w celu kompleksowej obsługi nagłówków IPv6 oraz protokołów routingu ICMPv6.